CompSci 471

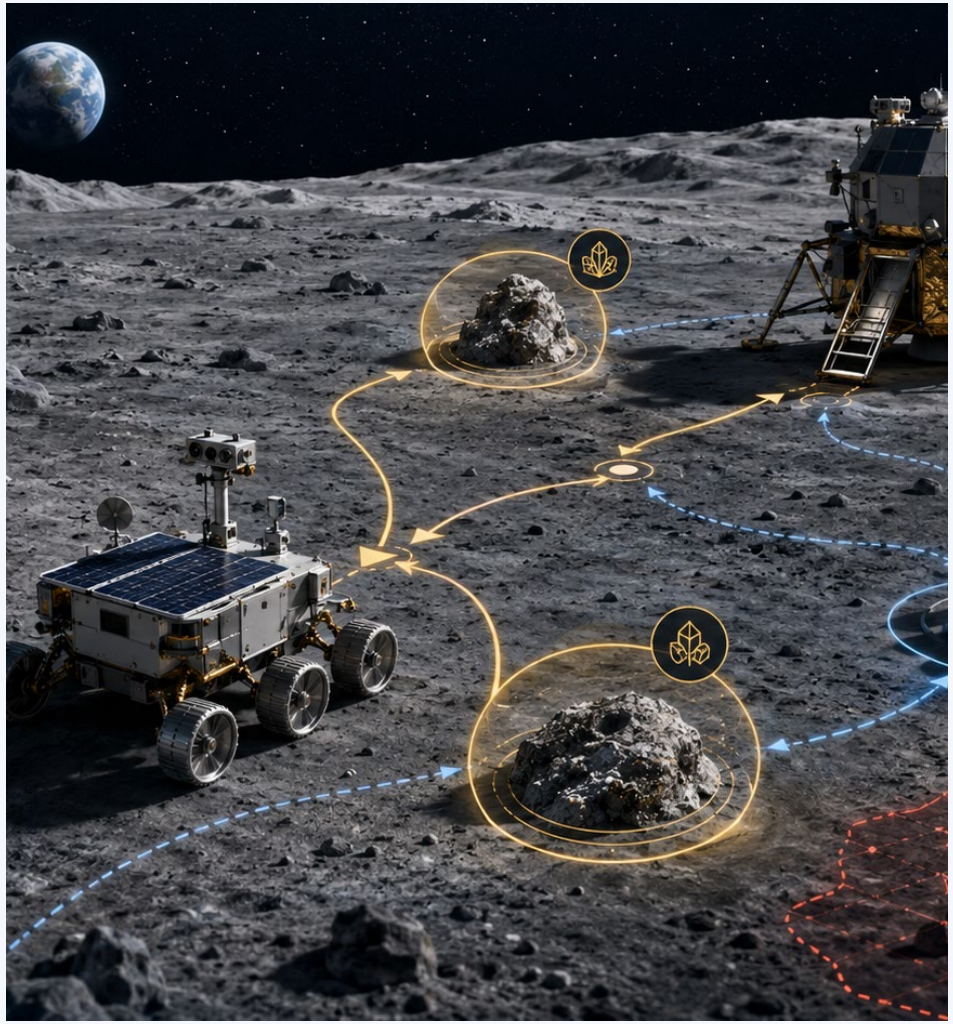
Artificial Intelligence

Lesson 6: Informed Search

Cost so far, estimate remaining

→ informed priority

Start **g(n)** **h(n)** **f(n)** **Goal**



A heuristic does not reveal the future. It gives search a defensible estimate of where to look next.

Fall 2026

Retrieval warm-up

UCS knows the cost already incurred, but not what remains.

Node	g(n)
A	3
B	6
C	4

Prompt

Which node does UCS select?

What does UCS know?

What does UCS not know?

Today we add the missing piece: an estimate of future cost.

Today's mission

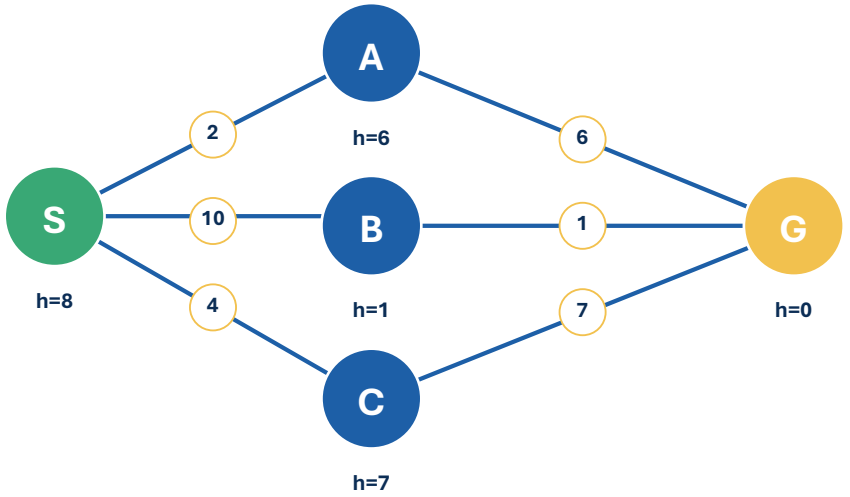
By the end of Lesson 6, students should be able to...

1. **Define heuristics and their role in search**
2. **Compare Greedy Search and A* Search**
3. **Explain how combining cost and heuristics affects search behavior**

Performance target: calculate g, h, and f; identify the next node for UCS, Greedy, and A*; explain why they differ.

Running scenario: relay recovery

Same waypoint problem. New information about what remains.



Edge labels = energy cost; h = estimated energy remaining.

Last lesson

- routes
- edge costs
- accumulated cost g(n)

New today

- remaining-cost estimate h(n)

How could an estimate help the rover avoid exploring blindly?

What is a heuristic?

A heuristic estimates the remaining cost from a state to a goal.

h(n) **estimated remaining cost from state n to a goal**

h(n) is...	h(n) is not necessarily...
an estimate	the exact remaining cost
based on problem knowledge	the cost already incurred
used to prioritize exploration	a guarantee of the best route
goal-directed information	a replacement for the goal test

Where do heuristics come from?

A heuristic usually solves a simplified version of the problem.

Road navigation

straight-line distance

Grid movement

Manhattan distance

Lunar rover

terrain-distance energy estimate

Puzzle

misplaced pieces

Cyber investigation

remaining effort estimate

Useful heuristics are informative enough to guide search and cheap enough to be worth computing.

Three quantities, three meanings

Informed search separates cost so far from estimated cost remaining.

$$f(n) = g(n) + h(n)$$

Quantity	Meaning	Direction
g(n)	exact cost from start to n	backward
h(n)	estimated cost from n to goal	forward
f(n)	estimated total cost through n	both

Same frontier, three questions

Different algorithms ask different questions about the same frontier.

Node	g(n)	h(n)	f(n)
A	3	7	10
B	6	2	8
C	4	5	9

UCS cheapest path so far — **A**

Greedy appears closest — **B**

A* lowest estimated total — **B**

Greedy Best-First Search

Greedy expands the frontier node with the lowest h(n).

Greedy priority = h(n)

Greedy ignores
g(n)
depth
insertion order

Question it asks:

“Which frontier node appears closest to the goal right now?”

Greedy frontier mechanics

Sort by the estimate of remaining cost.

Node	h(n)
D	1
B	3
A	5
C	7

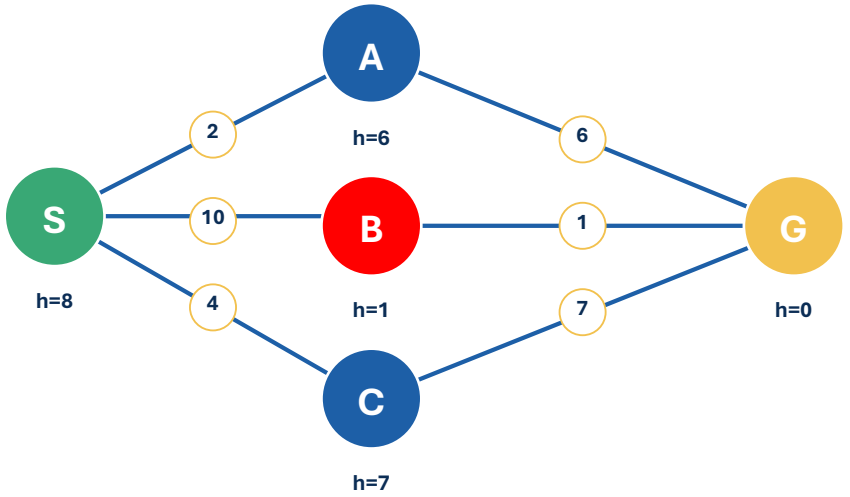
Greedy selects D

Lowest h(n) = 1

Not because it was added first.
Not because it is cheapest so far.

Guided Greedy trace

Watch what changes when the estimate drives the frontier.



Edge labels = energy cost; h = estimated energy remaining.

Step	Expanded	Frontier h(n)	Next
0	**S**	A:6, B:1, C:7	B
1	**B**	A:6, C:7, G:0	G
2	**G**	goal found	stop

Greedy heads toward the node that appears closest—even if reaching it was expensive.

Greedy can be shortsighted

A low h(n) can hide an expensive path already taken.

Route	g(n)	h(n)	g+h
A	2	6	8
B	10	1	11

Greedy selects B

because h(B)=1 is the smallest estimate remaining.

But A has a lower estimated total cost through the node.

Strengths and limitations of Greedy

Greedy is useful, but its guarantee is not the same as UCS or A*.

Strengths	Limitations
strongly goal-directed	ignores cost already incurred
can reach a goal quickly	can follow misleading estimates
may expand fewer nodes	does not generally guarantee least cost
simple frontier rule	quality depends heavily on h(n)

Greedy trusts the estimate.

A* balances past and future

A* expands the node with the smallest estimated total solution cost.

$$f(n) = g(n) + h(n)$$

g(n)

cost already paid

h(n)

estimated cost remaining

f(n)

estimated total through n

A* does not simply chase the closest-looking state. It asks what complete route looks cheapest.

A* frontier mechanics

Compute f(n), then select the smallest value.

Node	g(n)	h(n)	f(n)
P	2	7	9
Q	5	2	7
R	3	5	8
T	7	1	8

Selections

UCS → P

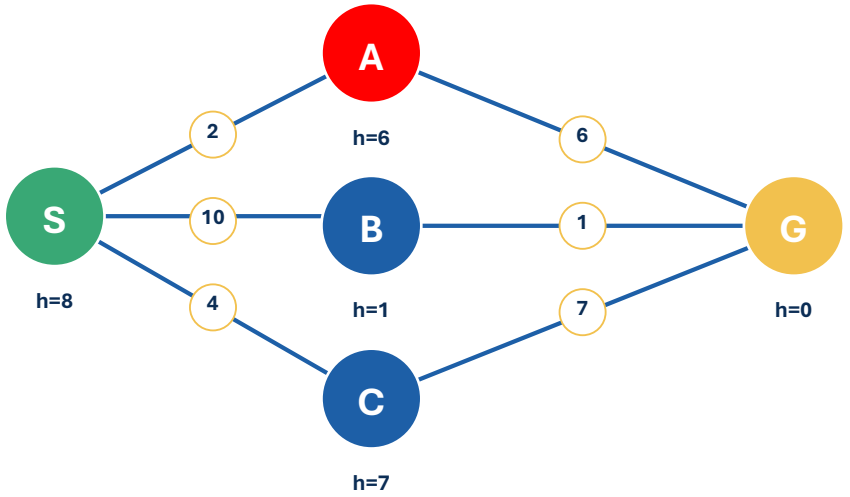
Greedy → T

A* → Q

Same frontier. Different priority rule.

Guided A* trace

A* may reject the smallest h if getting there was too expensive.



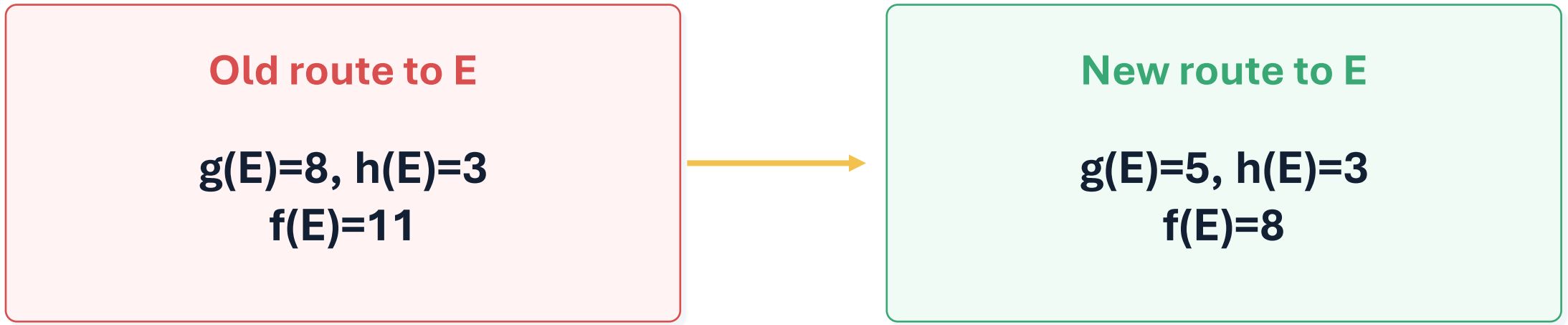
Edge labels = energy cost; h = estimated energy remaining.

Step	Expanded	Frontier f(n)	Next
0	**S**	A:8, B:11, C:11	A
1	**A**	G:8, B:11, C:11	G
2	**G**	goal found	stop

Greedy chose B; A* chooses A because 2 + 6 beats 10 + 1.

A cheaper path still replaces an expensive path

A* still tracks actual path cost.



Update g, recompute f, update parent, reorder the frontier.

Compare UCS, Greedy, and A*

Each algorithm answers a different frontier question.

Algorithm	Uses	Ignores	Guiding question
UCS	g(n)	h(n)	What has been cheapest so far?
Greedy	h(n)	g(n)	What looks closest to the goal?
A*	g(n)+h(n)	neither	What route appears cheapest overall?

Greedy trusts the estimate. UCS trusts the cost already paid. A* considers both.

Same frontier, different priorities

Name the number that controls the choice.

Node	g(n)	h(n)	f(n)
A	2	8	10
B	5	3	8
C	4	5	9

UCS

A

lowest g

Greedy

B

lowest h

A*

B

lowest f

Do not just pick the smallest number in the table. Pick the smallest number for the algorithm.

A* contains UCS as a special case

When the heuristic gives no guidance, A* behaves like UCS.

If h(n) = 0 for every node...

$$f(n) = g(n) + 0 = g(n)$$

A* with a zero heuristic uses the same priority as UCS.

A zero heuristic is unhelpful, but it is not misleading.

What makes a heuristic useful?

Guidance helps only if the estimate is informative and practical.

Heuristic behavior	Likely search effect
meaningful estimates	focuses exploration
identical values everywhere	provides little guidance
misleading values	sends search toward poor regions
cheap to compute	may save overall search effort
expensive to compute	may not justify its cost

Next question: how do we know whether one heuristic is safe or better than another?

Lesson 7

Algorithm-selection challenge

Choose the frontier rule that fits the available information and goal.

Scenario A

only accumulated fuel is available

UCS

Scenario B

any plausible goal is urgent; route cost is secondary

Greedy

Scenario C

cost so far and defensible remaining estimate are available

A*

Justify the answer by naming the information the algorithm uses.

Frontier sprint

Compute f(n), then compare UCS, Greedy, and A*.

Node	g(n)	h(n)	f(n)
J	4	5	9
K	7	1	8
L	3	6	9
M	5	2	7

Tasks

1. Compute f(n)
2. UCS selection
3. Greedy selection
4. A* selection
5. Explain disagreement

Diagnose the mistake

Correct the statement and name the confused quantity.

“h(n) is the cost from the start to the current node.”

“Greedy selects the node with the lowest g(n)+h(n).”

“A* ignores the route already traveled.”

Partner task: rewrite each statement so it is true.

Synthesis: informed search

Rebuild the formulas and priority rules from memory.

Complete

g(n) = _____

h(n) = _____

f(n) = _____

Match

UCS uses: _____

Greedy uses: _____

A* uses: _____

Greedy trusts the estimate. UCS trusts the cost already paid. A* considers both.

Exit Ticket

Apply the priority rules one more time.

Node	g(n)	h(n)	f(n)
A	3	6	9
B	5	2	7
C	2	7	9

Answer

1. Compute f(n)
2. Which node does UCS select?
3. Which node does Greedy select?
4. Which node does A* select?
5. Why does A* use both g and h?

HW1 due. Quiz 1 next class. Next: Heuristics — when is a heuristic safe, strong, or misleading?